

Experiment 12: Cluster Streaming Data Using Single-Pass K-Means Algorithm

Aim

Real time clustering of streaming data using incremental single-pass K-means.

Theory

K-means normally needs the entire dataset in memory and repeats the assign-and-update cycle many times until the centroids stop moving. This is not possible for a data stream, where records arrive continuously and each record can be read only once.

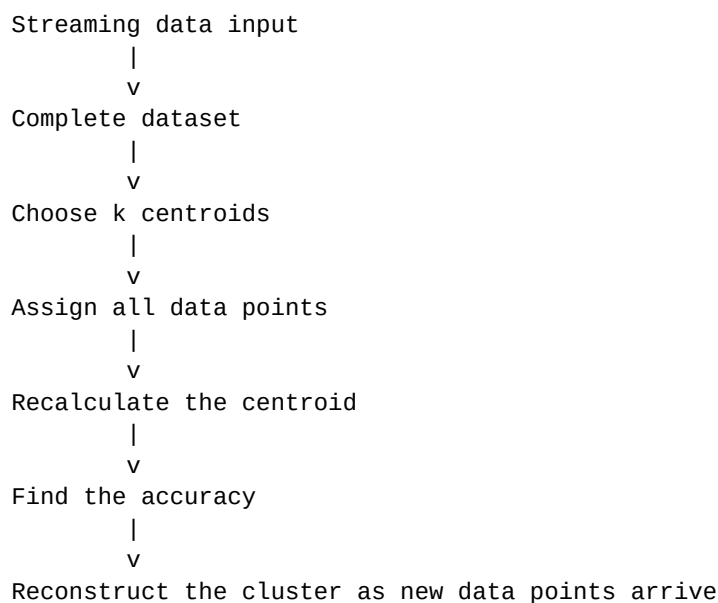
Single-pass K-means solves this by updating the centroids incrementally. The first k records become the initial centroids. When a new point arrives, the distance to every centroid is calculated and the point is assigned to the nearest one. That centroid is then moved towards the new point using a learning rate equal to the running mean:

```
learning_rate = 1 / count[cluster]
```

```
centroid = centroid + learning_rate * (x - centroid)
```

Only the centroid and the count are stored for each cluster, so the memory used stays constant no matter how long the stream runs, and every point is processed exactly once.

Data Pipeline



Procedural Steps

1. Generate or acquire streaming data.
2. Process one observation at a time.
3. Maintain k cluster centroids.
4. Assign every incoming data point.
5. Update the selected centroid immediately.
6. Avoid storing the complete stream.
7. Visualize cluster formation.

8. Measure the cluster quality.
9. Compare the streaming result with conventional K-Means.
10. Result analysis.

Requirements

- Python 3.x
- Jupyter Notebook
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Step 1: Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score
```

Interpretation: Imports the modules needed to generate the stream, hold the data, plot the clusters and measure the cluster quality.

Step 2: Generate Streaming Dataset

```
np.random.seed(42)

clusters_1 = np.random.normal(loc=(2, 2), scale=0.6, size=(300, 2))
clusters_2 = np.random.normal(loc=(8, 8), scale=0.7, size=(300, 2))
clusters_3 = np.random.normal(loc=(5, 2), scale=0.5, size=(300, 2))

# combining the cluster vertically
data = np.vstack((clusters_1, clusters_2, clusters_3))
np.random.shuffle(data)

# convert the data into a pandas dataframe
df = pd.DataFrame(data, columns=['feature1', 'feature2'])

print("first five observations")
print(df.head())
```

Interpretation: Three groups of 300 observations are generated around the centres (2, 2), (8, 8) and (5, 2). The observations are shuffled so that they arrive in random order, exactly like a real stream.

Step 3: Single-Pass K-Means Function

```
def single_pass_kmeans(stream, k):

    stream = np.asarray(stream)

    centroids = stream[:k].copy()
    counts = np.ones(k, dtype=int)

    labels = []
    centroid_history = [centroids.copy()]

    for x in stream[k:]:
```

```

distances = np.linalg.norm(
    centroids - x,
    axis=1
)

# Find the nearest centroid
cluster = np.argmin(distances)

counts[cluster] += 1

learning_rate = 1.0 / counts[cluster]

centroids[cluster] = (
    centroids[cluster]
    + learning_rate * (x - centroids[cluster])
)

# Store cluster assignment
labels.append(cluster)
centroid_history.append(centroids.copy())

return (
    np.array(labels),
    np.array(centroid_history),
    counts
)

```

Interpretation: The first k observations become the initial centroids. Every remaining observation is read exactly once, assigned to the nearest centroid by Euclidean distance, and then discarded. The learning rate $1 / \text{count}$ makes the update equal to the running mean of that cluster, so the complete stream is never stored.

Step 4: Execute the Algorithm

```

# select the number of clusters
k = 3

# execute the single-pass K-means algorithm
labels, centroid_history, counts = single_pass_kmeans(df.values, k)

final_centroids = centroid_history[-1]

```

Interpretation: Runs the algorithm over the stream and keeps the last set of centroids as the final result.

Step 5: Display the Final Centroids

```

print("Final Centroids:")

for i, centroid in enumerate(final_centroids):
    print(
        f"Cluster {i}: "
        f"Feature_1={centroid[0]:.3f}, "
        f"Feature_2={centroid[1]:.3f}"
    )

```

Interpretation: Displays the position of each centroid after the whole stream has been processed.

Step 6: Display the Cluster Sizes

```

print("Cluster counts:")

```

```

for i, count in enumerate(counts):
    print(
        f"Cluster {i}: {count} observations"
    )

```

Interpretation: Shows how many observations were assigned to each cluster.

Step 7: Measure the Cluster Quality

```

initial_labels = []

for i in range(k):
    distances = np.linalg.norm(
        final_centroids - data[i],
        axis=1
    )
    initial_labels.append(
        np.argmin(distances)
    )

all_labels = np.concatenate([
    np.array(initial_labels),
    labels
])

score = silhouette_score(
    data,
    all_labels
)

print(f"\nSilhouette_score: {score:.4f}")

```

Interpretation: The first k observations were used to initialize the centroids, so they were never labelled inside the loop. They are labelled here against the final centroids. The silhouette score measures how well each observation fits its own cluster compared to the nearest other cluster, on a scale from -1 to 1.

Step 8: Visualize Cluster Formation

```

plt.figure(figsize=(9, 6))

for cluster_id in range(k):

    cluster_points = data[
        all_labels == cluster_id
    ]

    plt.scatter(
        cluster_points[:, 0],
        cluster_points[:, 1],
        label=f"Cluster {cluster_id}",
        alpha=0.6
    )

# Plot the final centroids
plt.scatter(
    final_centroids[:, 0],
    final_centroids[:, 1],
    marker="x",
    s=250,
    label="centroids"
)

```

```
)

# Add the features for the plot
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Single Pass K Means")
plt.legend()
plt.grid(True)

plt.show()
```

Interpretation: Plots every observation coloured by its cluster, with the final centroids marked by crosses.

Step 9: Compare with Conventional K-Means

```
from sklearn.cluster import KMeans

def wcss(points, centroids, cluster_labels):
    total = 0
    for i in range(len(centroids)):
        cluster_points = points[cluster_labels == i]
        total += np.sum((cluster_points - centroids[i]) ** 2)
    return total

wcss_stream = wcss(data, final_centroids, all_labels)

kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
kmeans.fit(data)

score_batch = silhouette_score(data, kmeans.labels_)
wcss_batch = kmeans.inertia_
```

Interpretation: Conventional K-Means stores the whole dataset and repeats the assign-and-update cycle until convergence, so it is used here as the reference against which the single-pass result is judged.

Step 10: Result Analysis

```
print("Observations processed :", len(data))
print("Observations stored      :", 0)
print("Values kept in memory    :", k, "centroids and", k, "counts")

print("Passes over the data")
print("  Single-Pass K-Means   : 1")
print("  Conventional K-Means   :", kmeans.n_iter_, "iterations")
```

Interpretation: Compares the cost of the two methods in terms of memory and passes over the data.

Sample Input

No external input file is required. The stream is generated internally:

```
Group 1 : 300 observations, centre (2, 2), scale 0.6
Group 2 : 300 observations, centre (8, 8), scale 0.7
Group 3 : 300 observations, centre (5, 2), scale 0.5

Total   : 900 observations, shuffled
k       : 3
seed    : 42
```

Expected Output

```
first five observations
  feature1 feature2
0  9.712026  8.090455
1  5.608579  2.760658
2  7.571048  7.728609
3  1.908929  2.352990
4  5.245487  2.367439
```

Final Centroids:

Cluster 0: Feature_1=8.529, Feature_2=8.482

Cluster 1: Feature_1=3.503, Feature_2=2.022

Cluster 2: Feature_1=7.760, Feature_2=7.752

Cluster counts:

Cluster 0: 123 observations

Cluster 1: 600 observations

Cluster 2: 177 observations

Silhouette_score: 0.5661

A scatter plot of the three clusters with the final centroids marked by crosses is then displayed.

Comparison with Conventional K-Means

Method	Silhouette	WCSS	Passes
Single-Pass K-Means	0.5661	1931.72	1
Conventional K-Means	0.7275	643.23	2 iterations

Conventional centroids: (8.075, 8.051), (5.009, 2.057) and (1.987, 1.988).

Interpretation

- A stream of 900 observations belonging to three natural groups is generated and shuffled.
- The first 3 observations become the initial centroids.
- Each remaining observation is read once, assigned to the nearest centroid and then discarded.
- The centroid is corrected by the learning rate $1 / \text{count}$, which is the running mean.
- Only k centroids and k counts are stored, so the memory does not grow with the stream.
- The final centroids are (8.529, 8.482), (3.503, 2.022) and (7.760, 7.752), with cluster sizes of 123, 600 and 177.
- With the seed 42 the first three observations happened to include two points from the region around (8, 8), so two centroids started in the same group. As a result the groups near (2, 2) and (5, 2) were merged into a single cluster of 600 observations.
- This shows the main limitation of single-pass K-Means: the result depends strongly on which observations arrive first, because there is no second pass to correct a poor initialization.
- The silhouette score of 0.5661 still indicates reasonably separated clusters.
- Conventional K-Means recovers all three true centres and reaches a higher silhouette score of 0.7275 with a much lower WCSS, but it needs the whole dataset in memory and several passes over it.

Result

The streaming data was successfully clustered in real time using the incremental single-pass K-Means algorithm. Each observation was processed exactly once, the selected centroid was updated immediately using the learning rate $1 / \text{count}$, and the complete stream was never stored. The cluster quality was measured using the silhouette score and compared with conventional K-Means, showing that the single-pass method clusters a stream in one pass with constant memory, at the cost of being sensitive to the order in which the observations arrive.